Digital VLSI Design

DVD Assignment

4-bit Array Multiplier from RTL to GDSII with Open-Source Tools

SkyWater SKY130 (130 nm), standard cell library `sky130_fd_sc_hd`

Submitted by

Nandish P Roll No. 2026203002

Dheerendra Singh Roll No. 2026203017

GitHub Repository

`https://github.com/Hatakekakashi210/mult4_2026203002`

Submitted to

Prof. Zia Abbas

Contents

1 Introduction **1**

2 Design and Verification (Part A) **1**

2.1 Design 1

2.2 Verification 3

2.3 Worked example and waveform 4

3 Synthesis (Part B) **6**

4 Timing (Part C) **7**

5 Physical Design (Part D) **9**

6 Answers to Questions **11**

7 Conclusion **13**

List of Figures

1 Block diagram of `mult_array`: 8 input flip-flops, 16 AND gates, 12 full adders (three rows of four) and 8 output flip-flops (16 flip-flops in total). The AND array also supplies pp_2 and pp_3 to rows 2 and 3. The low product bits $p[0]$, $p[1]$, $p[2]$ are finished in the AND array and rows 1 and 2. 2

2 Testbench summary line for the correct design: `PASS`, 0 errors. 3

3 Error test with `sum = a^b`: the testbench prints `FAIL` with 95 errors. 4

4 Handwritten binary multiplication of $A \times B = 12 \times 7$ ($A = 1100_2$, $B = 0111_2$): the four partial products, their shifts, and the sum $P = 01010100_2 = 84$. . . . 5

5 GTKWave waveform of `clk`, `rst_n`, `a`, `b` and `p` for 12×7 (decimal), with the 2-cycle latency marked. 5

6 Yosys statistics report (65 cells, 746.966 µm^2) and the PASS line of the gate-level simulation. 6

7 OpenSTA worst slack at $T = 10$ ns and at $T = 2.59$ ns. 7

8 Critical path report (clock group) at $T = 2.59$ ns. 8

9 LibreLane attempt 3 (6.5 ns): the flow completes with Antenna, LVS and DRC passed. 9

10 Final layout (GDSII) of `mult_array` in KLayout: the whole 150 µm × 150 µm die with power stripes, the standard cell area and the I/O pins. 11

List of Tables

1 Tools and versions used. 1
2 Part B results (Yosys, `sky130_fd_sc_hd`, typical corner). 6
3 Part C results (OpenSTA, typical corner). 7
4 Cells on the critical path at $T = 2.59\,\text{ns}$ (from `mult_array_timing.rpt`). . . . 8
5 LibreLane attempts. 9
6 Part D results (final run, attempt 3). 10
7 Cell classes in the final layout (from `metrics.json`). 10
8 Scaling from 4 bits to 8 bits (estimate for 8 bits). 12

1 Introduction

This report documents the design of a 4-bit × 4-bit unsigned array multiplier (8-bit product) in Verilog and its complete open-source RTL-to-GDSII flow on the SkyWater SKY130 process: RTL simulation (Part A), logic synthesis with Yosys (Part B), static timing analysis with OpenSTA (Part C), and place and route with LibreLane to a DRC- and LVS-clean layout (Part D). The multiplier registers its inputs and its output (16 flip-flops, latency of 2 clock cycles) and is built from 16 AND gates and 12 `full_adder` instances. All tools ran inside the IIC-OSIC-TOOLS Docker image; the versions are listed in Table 1. The complete code and run results can be found in our GitHub repository at `https://github.com/YOUR-USERNAME/mult4_2026203017`, which strictly follows the file architecture of the assignment.

Table 1: Tools and versions used.

Tool	Version
Icarus Verilog	14.0 (devel)
Yosys	0.69
OpenSTA	3.1.0
LibreLane	v3.1.0.dev3
KLayout	0.30.12
Docker image	`hpretl/iic-osic-tools` (IIC-OSIC-TOOLS)
PDK / cell library	`sky130A` / `sky130_fd_sc_hd` (typical corner for Parts B and C)

2 Design and Verification (Part A)

2.1 Design

`full_adder` is written with `assign` statements (`sum` $= a \oplus b \oplus c_{in}$, `cout` $= ab + bc_{in} + c_{in}a$). `mult_array` has ports `clk`, `rst_n` (asynchronous, active low), `a[3:0]`, `b[3:0]` and `p[7:0]`. It registers `a` and `b`, forms the partial products $pp_i[j] = a[j] \,\&\, b[i]$, adds them in three rows of four full adders as in Section 2 of the assignment (12 `full_adder` instances, no `*` operator), and registers the 8-bit result into `p`. Fig. 1 shows the structure from the inputs (top) to the output (bottom).

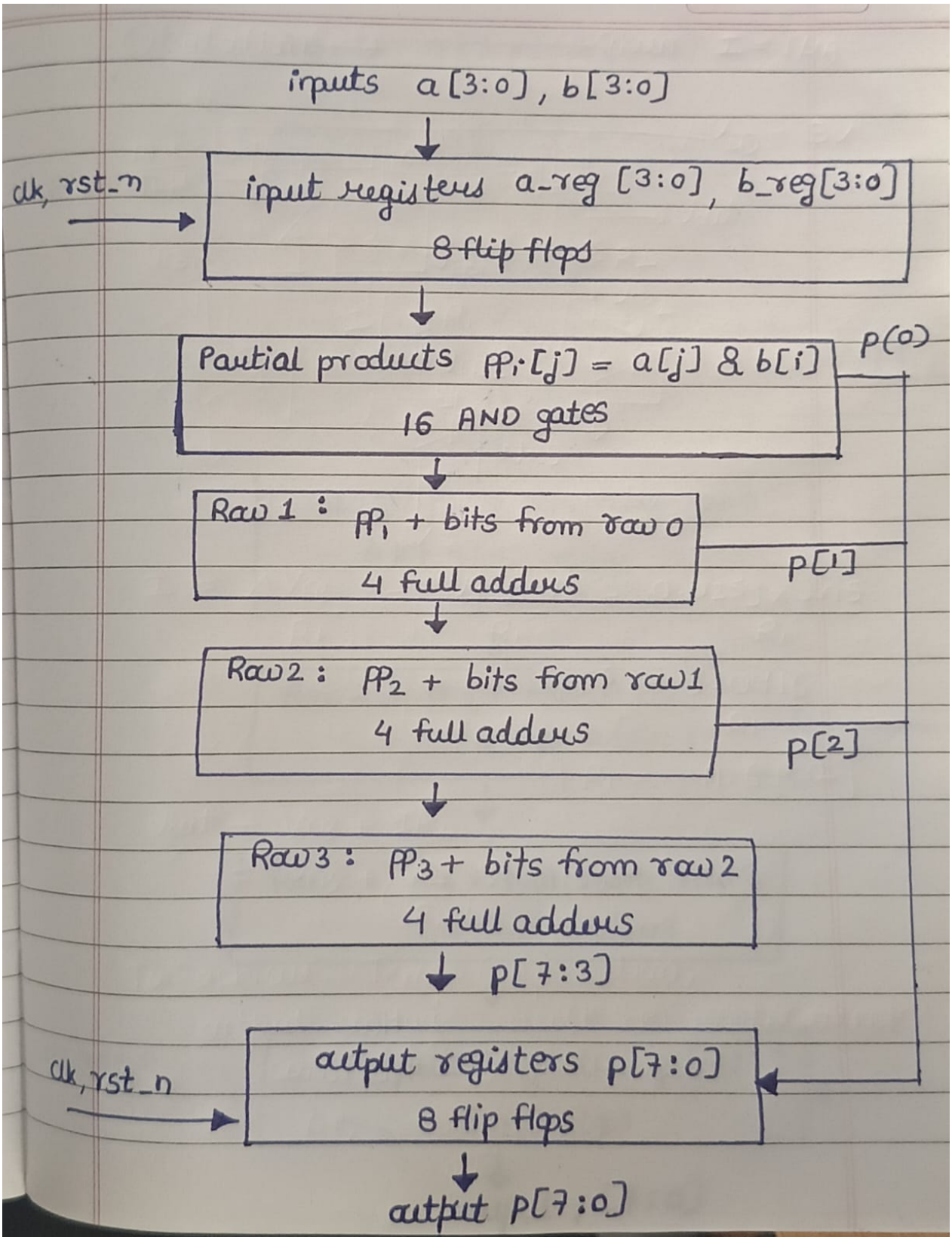


Figure 1: Block diagram of `mult_array`: 8 input flip-flops, 16 AND gates, 12 full adders (three rows of four) and 8 output flip-flops (16 flip-flops in total). The AND array also supplies pp_2 and pp_3 to rows 2 and 3. The low product bits $p[0]$, $p[1]$, $p[2]$ are finished in the AND array and rows 1 and 2.

2.2 Verification

The testbench `tb_mult` generates a 10 ns clock, holds `rst_n` low for the first 2 cycles, applies all 256 combinations of `a` and `b`, waits for the 2-cycle latency, compares `p` with `a * b`, prints every mismatch and exactly one summary line, and writes `mult.vcd`. The design prints `PASS: 256 vectors checked, 0 errors` (Fig. 2).

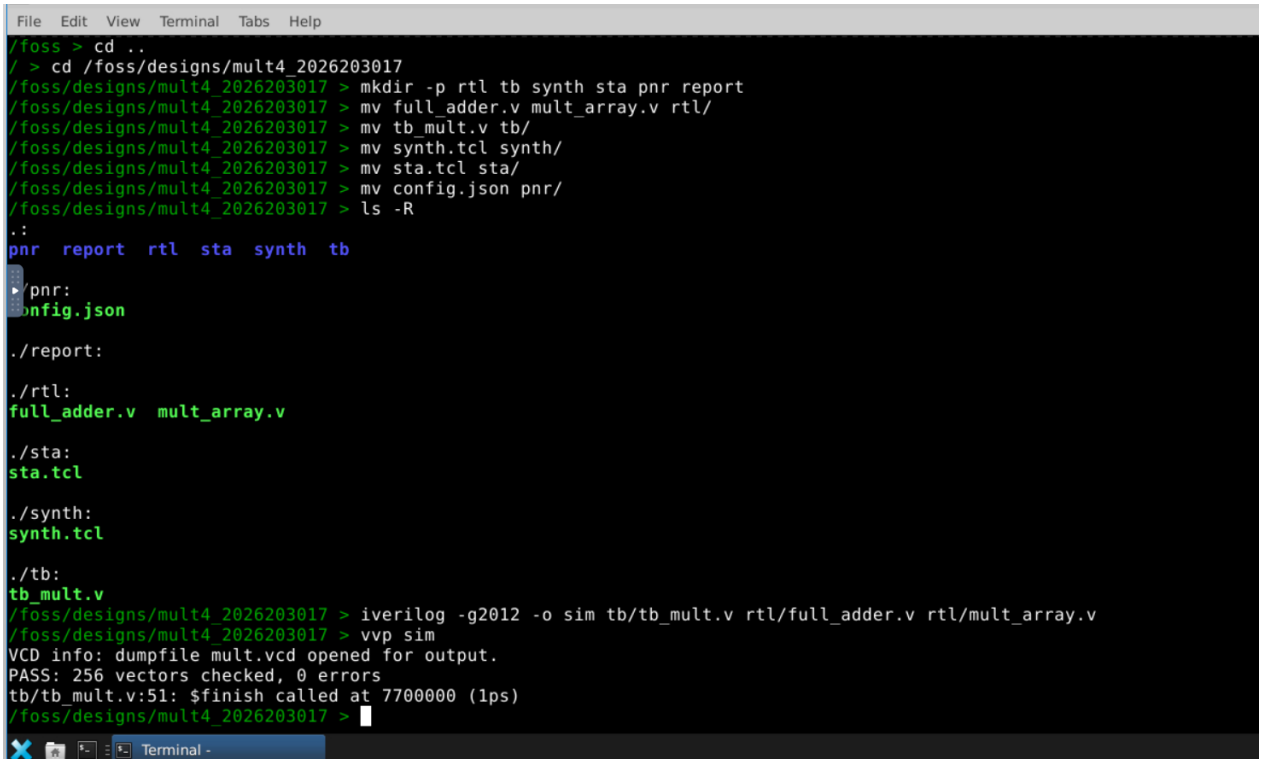


Figure 2: Testbench summary line for the correct design: `PASS`, 0 errors.

To check that the testbench catches errors, `sum` in `full_adder.v` was temporarily changed to `a^b`; the testbench then printed `FAIL: 256 vectors checked, 95 errors` (Fig. 3), and the change was undone.

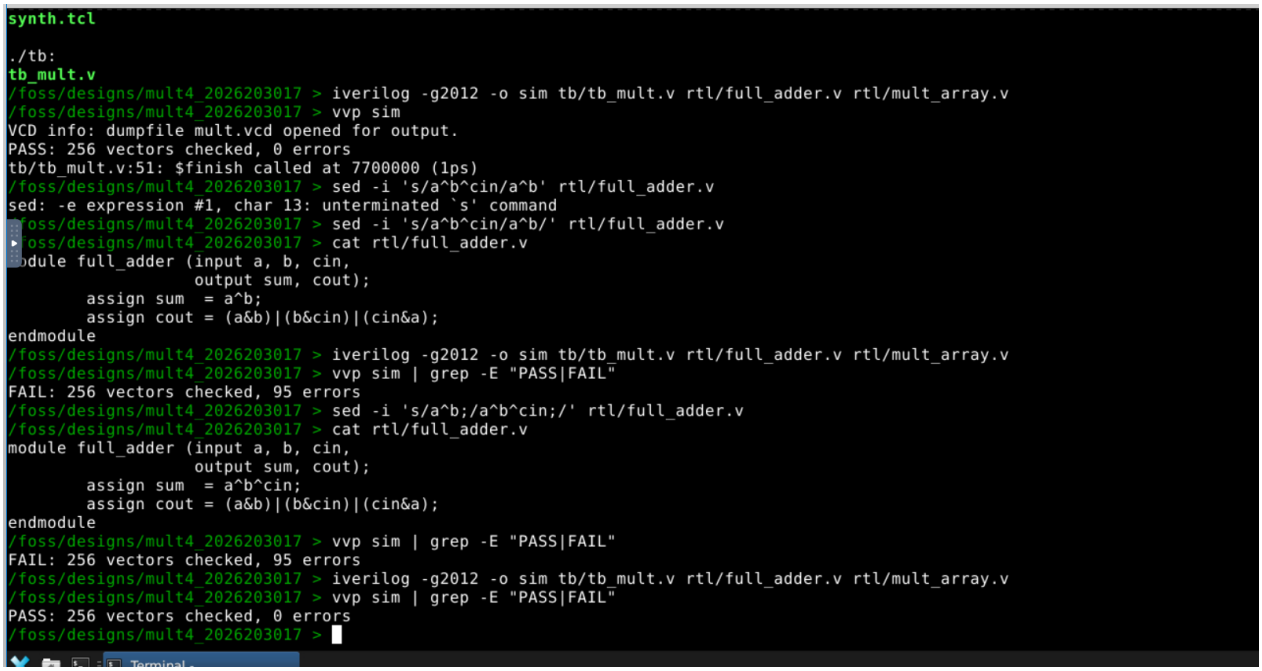


Figure 3: Error test with `sum = a^b`: the testbench prints `FAIL` with 95 errors.

2.3 Worked example and waveform

The example uses the roll number 2026203017 (Dheerendra Singh): last digit 7, so $A = 7 + 5 = 12 = 1100_2$ and $B = 7 = 0111_2$ (since $A \neq 7$). Fig. 4 gives the handwritten binary multiplication of 12×7 in the layout of Section 2 of the assignment, and Table **??** gives the same calculation typed: $12 + 24 + 48 + 0 = 84 = 01010100_2$.

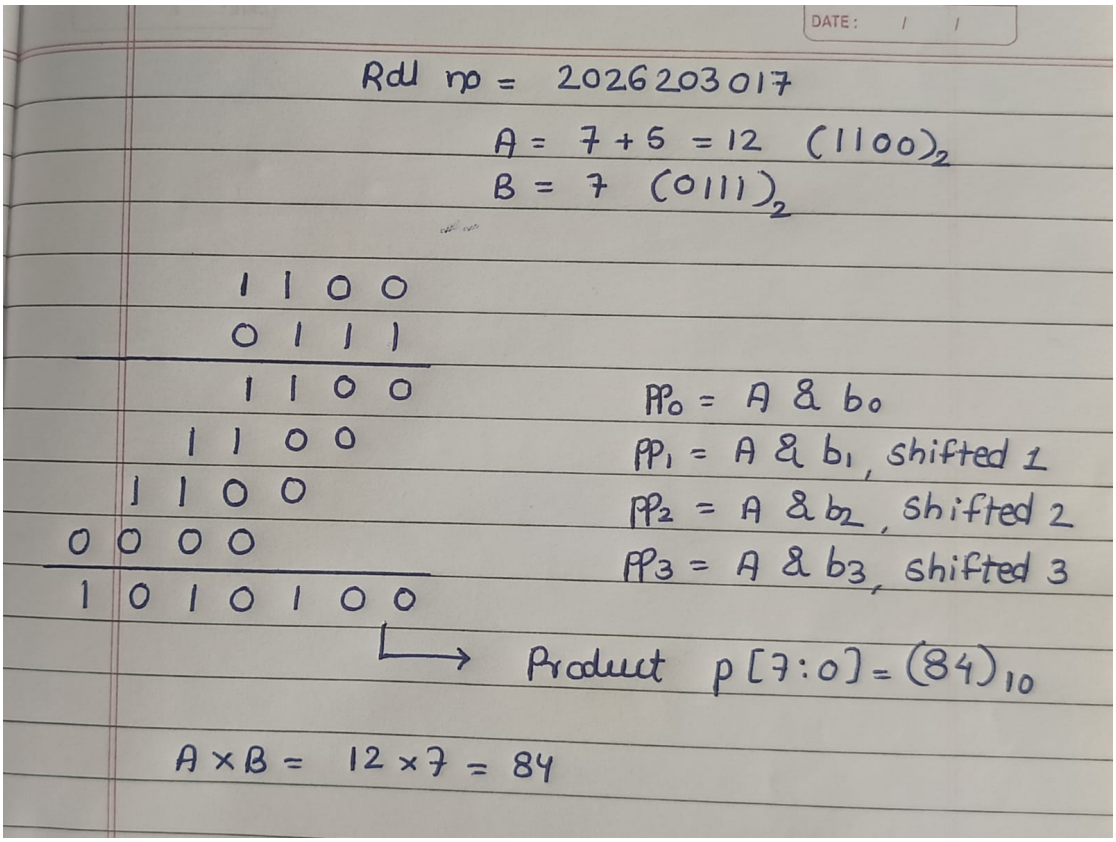


Figure 4: Handwritten binary multiplication of $A \times B = 12 \times 7$ ($A = 1100_2$, $B = 0111_2$): the four partial products, their shifts, and the sum $P = 01010100_2 = 84$.

In the waveform (Fig. 5), `a` = 12 and `b` = 7 are applied at 6000 ns. The input registers capture them at the rising edge at 6005 ns, and `p` becomes 84 at the next rising edge at 6015 ns, which is the latency of 2 clock cycles.

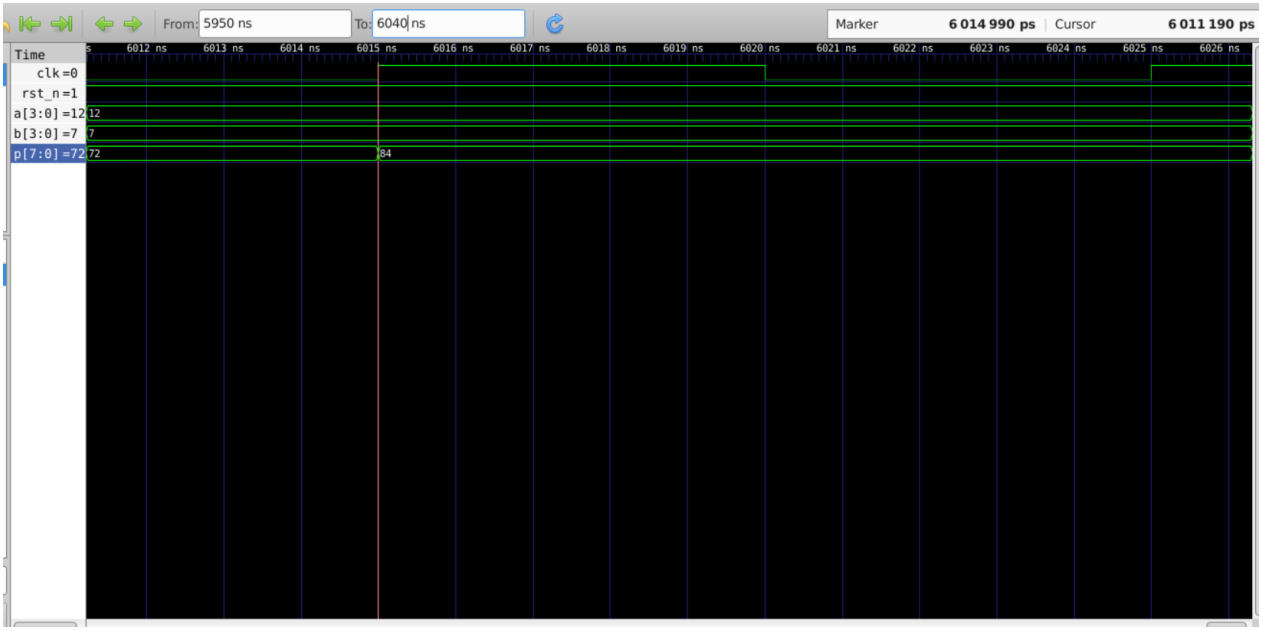


Figure 5: GTKWave waveform of `clk`, `rst_n`, `a`, `b` and `p` for 12×7 (decimal), with the 2-cycle latency marked.

3 Synthesis (Part B)

`synth/synth.tcl` reads `full_adder.v` and `mult_array.v`, runs `synth -top mult_array -flatten`, maps flip-flops (`dfflibmap`) and logic (`abc`) to the typical-corner SKY130 library, runs `opt_clean -purge`, writes the statistics report, and writes the netlist. Table 2 gives the results. The synthesized netlist was simulated with the same testbench and the SKY130 cell models and prints `PASS: 256 vectors checked, 0 errors`, so synthesis did not change the behaviour (Fig. 6).

Table 2: Part B results (Yosys, `sky130_fd_sc_hd`, typical corner).

Metric	Value
Total number of cells	65
Number of flip-flops (`dfrtp_1`)	16
Total cell area	746.966 µm^2
Flip-flop area (16 × 25.024 µm^2)	400.384 µm^2
Flip-flop area (% of total)	53.60 %
3 most-used cell types	`dfrtp_1` × 16, `nand2_1` × 12, `xnor2_1` × 5
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

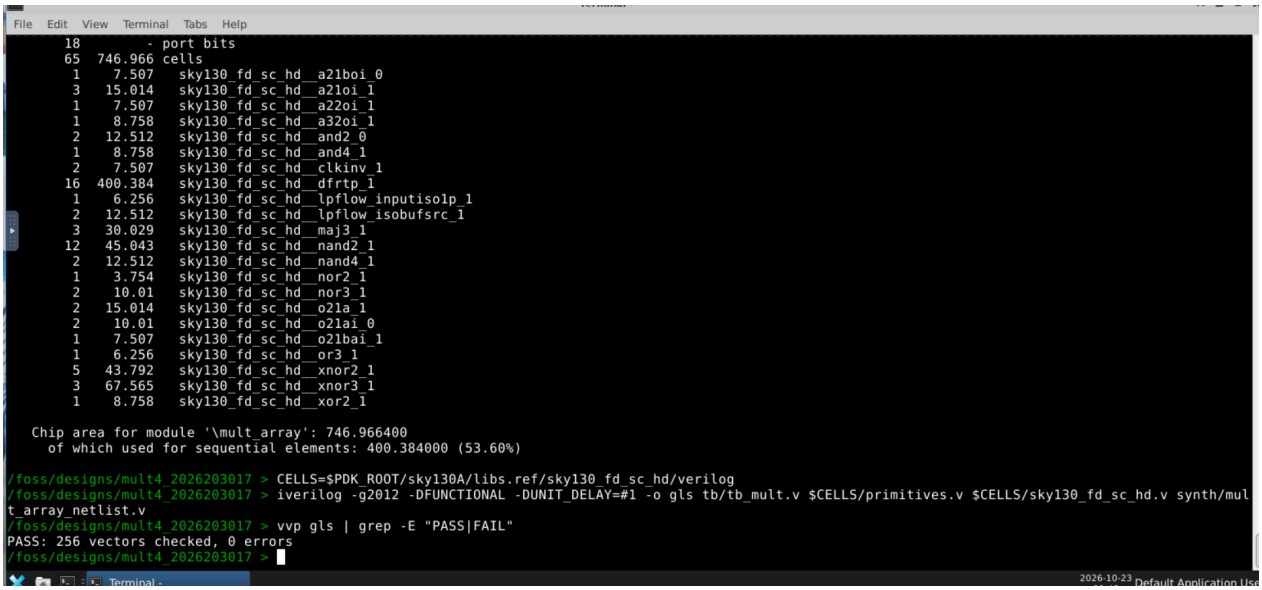


Figure 6: Yosys statistics report (65 cells, 746.966 µm^2) and the PASS line of the gate-level simulation.

4 Timing (Part C)

`sta/sta.tcl` reads the typical-corner library (`tt_025C_1v80`) and the netlist, defines a clock `clk` of period T, sets 1.0 ns input and output delays, and reports the worst setup slack and the critical path. With $T = 10$ ns the worst slack was 7.415 ns (printed as 7.42), so $T_{min} = 10 - 7.415 = 2.585$ ns, rounded to **2.59 ns**. Re-running at $T = 2.59$ ns gave a slack of +0.005 ns (Table 3, Figs. 7 and 8).

Table 3: Part C results (OpenSTA, typical corner).

Metric	Value
Worst slack at $T = 10$ ns	7.415 ns
T_{min}	2.59 ns
$F_{max} = 1000/T_{min}$	386 MHz
Slack at $T = T_{min}$	+0.005 ns (met)
Critical path start point	`_102_` (input register `b_reg[0]`)
Critical path end point	`_096_` (output register `p[6]`)
Number of cells on the critical path	8 logic cells (10 including the two flip-flops)
Latency $= 2 \times T_{min}$	5.18 ns
Throughput	386 million results/s

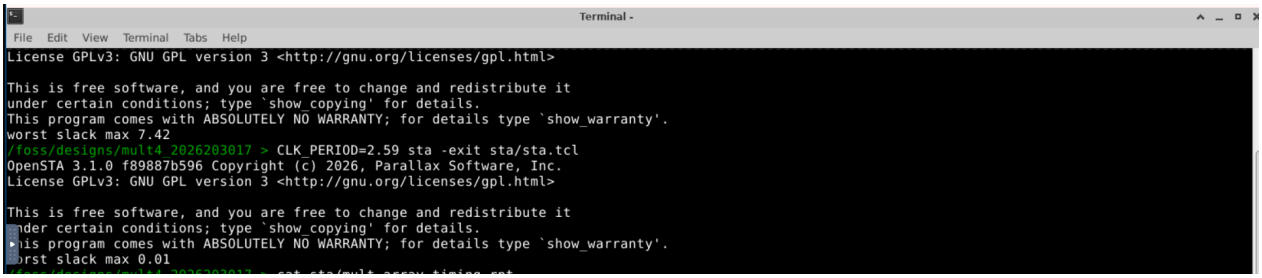


Figure 7: OpenSTA worst slack at $T = 10$ ns and at $T = 2.59$ ns.

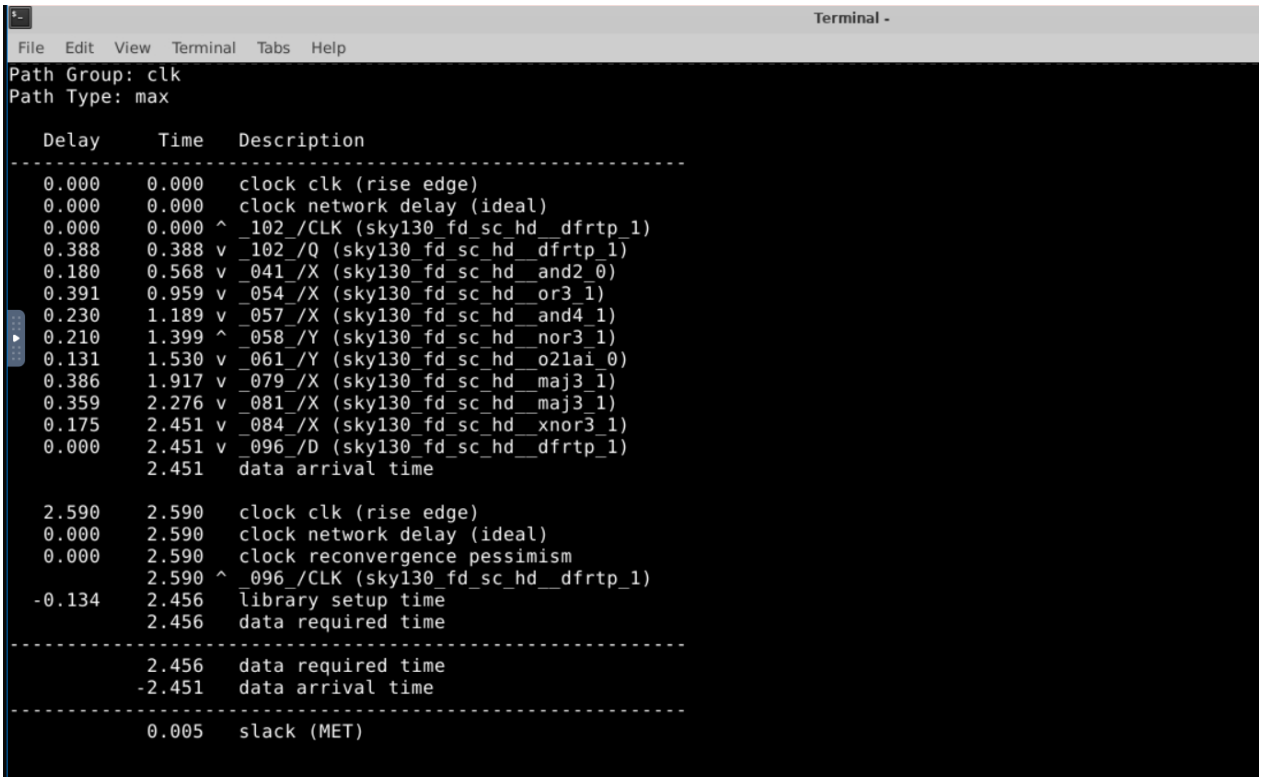


Figure 8: Critical path report (clock group) at $T = 2.59\,\text{ns}$.

Table 4: Cells on the critical path at $T = 2.59\,\text{ns}$ (from `mult_array_timing.rpt`).

#	Instance	Cell	Delay (ns)	Arrival (ns)	Role
start	`_102_`	`dfrtp_1` (clk→Q)	0.388	0.388	input register
1	`_041_`	`and2_0`	0.180	0.568	partial-product AND
2	`_054_`	`or3_1`	0.391	0.959	merged adder logic
3	`_057_`	`and4_1`	0.230	1.189	merged adder logic
4	`_058_`	`nor3_1`	0.210	1.399	merged adder logic
5	`_061_`	`o21ai_0`	0.131	1.530	merged adder logic
6	`_079_`	`maj3_1`	0.386	1.917	full-adder carry
7	`_081_`	`maj3_1`	0.359	2.276	full-adder carry
8	`_084_`	`xnor3_1`	0.175	2.451	full-adder sum
end	`_096_`	`dfrtp_1` (D pin)	–	2.451	output register

The path has 8 logic cells. Three of them are full-adder cells (two `maj3` carries and one `xnor3` sum), one is a partial-product AND, and four are gates in which Yosys merged the earlier ripple stages into complex gates. This is compared with the estimate of about $2N = 8$ full-adder delays in the answer to Question 2.

5 Physical Design (Part D)

`pnr/config.json` sets `DESIGN_NAME` = `mult_array`, the two Verilog files, `CLOCK_PORT` = `clk`, `FP_SIZING` = `absolute` and `DIE_AREA` = [0, 0, 150, 150]. The starting `CLOCK_PERIOD` was 1.2×2.59 ns rounded up to 3.5 ns. LibreLane checks setup timing in all corners, including the slow corner `ss_100C_1v60`, so the period had to be raised; the three attempts are in Table 5. After attempt 1 (slack -2.05 ns) the period was raised to 6.0 ns, which still failed by 0.119 ns, and then to 6.5 ns, which passed.

Table 5: LibreLane attempts.

Attempt	Run folder	`CLOCK_PERIOD`	Worst setup slack	DRC / LVS	Outcome
1	`RUN_2026-10-23_14-26-28`	3.5 ns	-2.050 ns	0 / 0	failed (setup, `ss_100C_1v60`)
2	`RUN_2026-10-23_14-47-42`	6.0 ns	-0.119 ns	0 / 0	failed (setup, `ss_100C_1v60`)
3	`RUN_2026-10-23_15-13-05`	6.5 ns	$+1.519$ ns	0 / 0	passed

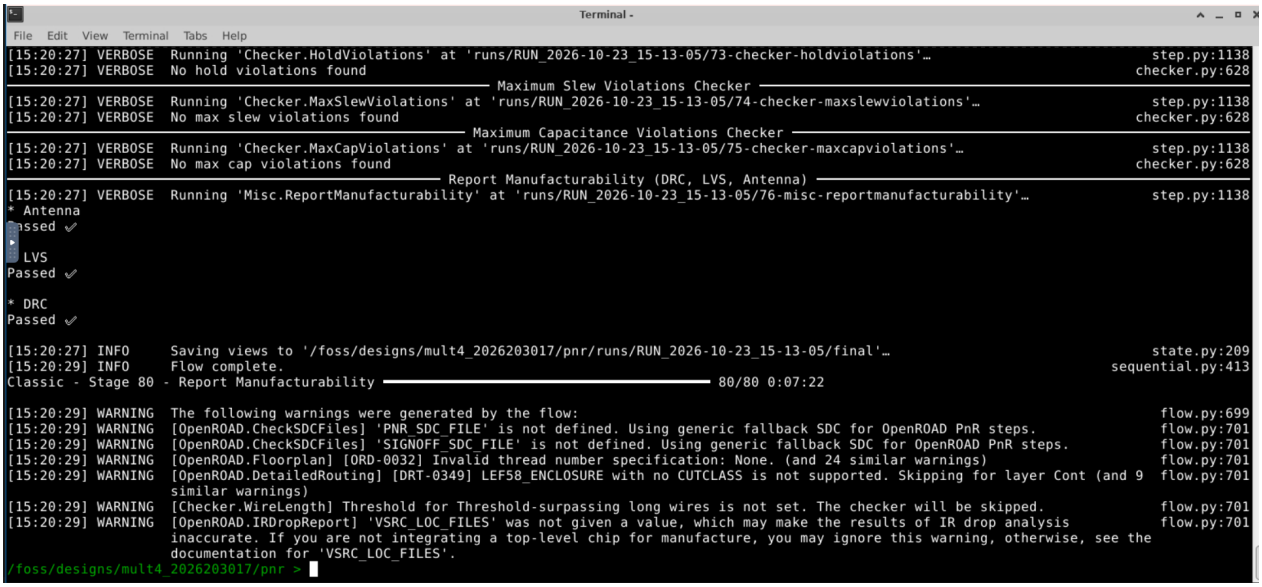


Figure 9: LibreLane attempt 3 (6.5 ns): the flow completes with Antenna, LVS and DRC passed.

Table 6: Part D results (final run, attempt 3).

Metric	Value
`CLOCK_PERIOD` used	6.5 ns
Number of runs needed	3
Core area (die area)	16199 µm^2 (22500 µm^2)
Cells after synthesis (Part B)	65
Cells after place and route	1317 in total (114 excluding fill cells)
Kinds of cells added	35 timing-repair buffers (16 hold buffers), 5 clock buffers, 1203 fill cells
Wirelength (detailed routing)	2803 µm (global-route estimate 4377 µm)
Setup slack (worst over all corners)	+1.519 ns
Hold slack (worst)	+0.106 ns
Post-route $F_{max} = 1000/(6.5 - 1.519)$	200.8 MHz
F_{max} from Part C	386 MHz
DRC errors (route, Magic, KLayout)	0, 0, 0
LVS errors	0

Table 7: Cell classes in the final layout (from `metrics.json`).

Class	Count
Flip-flops (sequential)	16
Inverters	3
Multi-input logic cells	55
Timing-repair buffers (including 16 hold buffers)	35
Clock-tree buffers	5
Standard cells (total)	114
Fill cells	1203
Total instances	1317

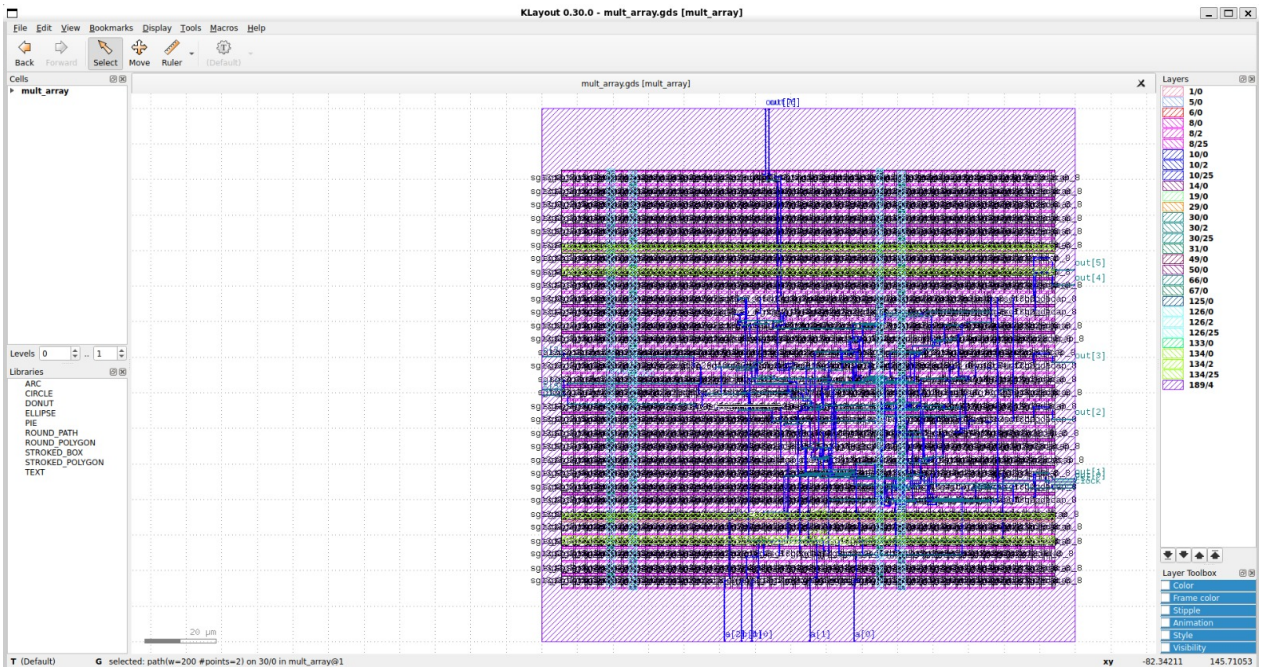


Figure 10: Final layout (GDSII) of `mult_array` in KLayout: the whole 150 µm × 150 µm die with power stripes, the standard cell area and the I/O pins.

6 Answers to Questions

Q1. Cells for the logic. Synthesis produced $65 - 16 = 49$ cells for the logic, not $16 + 12 = 28$. A full adder is not one library cell: its sum is a three-input XOR and its carry a majority function, so each adder needs several gates, or cells such as `xnor3_1` and `maj3_1` plus supporting gates. After `-flatten`, ABC also restructures the whole array: it merges the 16 AND gates into NAND/NOR and AND-OR-INVERT cells (12 `nand2_1`, plus `a21oi_1`, `a22oi_1`, `a32oi_1`, `o21ai_0`) and shares logic between neighbouring adders. In addition, the first adder of each row has a carry-in of 0 and the first row passes a 0 into row 1, so those adders reduce to half adders or disappear.

Q2. Critical path. The critical path runs from the input register `b_reg[0]` (`_102_`) to the output register bit `p[6]` (`_096_`). It contains 8 logic cells (Table 4): a clock-to-Q delay of 0.388 ns, 2.063 ns of logic and a setup time of 0.134 ns give $T_{min} = 2.585$ ns. The estimate of about $2N = 8$ full-adder delays is a structural worst case where each adder is one block. Here the path has about 8 gate delays, but only 3 of its cells are full-adder cells (two `maj3_1` and one `xnor3_1`); ABC merged the earlier ripple stages into complex gates, so one cell is about 0.26 ns on average instead of a whole full-adder delay per stage.

Q3. Flip-flop share of the area. Flip-flops take 400.384/746.966 = 53.60 % of the cell area. One `dfrtp_1` is 25.024 µm^2, about 6.7 times a `nand2_1` (3.754 µm^2) and 3.5 times the average logic cell (346.582/49 = 7.07 µm^2). The design is tiny: it has only 49 logic cells, but it needs 16 flip-flops to register 8 input bits and 8 output bits, and these asynchronous-reset flip-flops are large cells. This register cost is fixed, whereas the combinational logic of a 4-bit

multiplier is small.

Q4. Post-route F_{max}. The post-route F_{max} of 200.8 MHz is lower than the 386 MHz of Part C for these reasons. First, signoff checks the slow corner `ss_100C_1v60` (low voltage, high temperature), where cells are much slower than in the typical corner of Part C; at 3.5 ns the slow corner had a slack of −2.05 ns (about 5.55 ns, 180 MHz) while the typical corner had +0.57 ns (about 2.93 ns, 341 MHz). Second, after routing the wires add resistance and capacitance, which Part C does not include. Third, the clock tree has real delay and skew instead of the ideal clock of Part C. The flow added 35 timing-repair buffers (16 of them hold buffers, which fix hold violations and also add delay), 5 clock buffers (balance the clock arrival at the 16 flip-flops) and 1203 fill cells (fill the empty sites in the rows so that wells and power rails are continuous and density rules are met; they have no logic function). Well-tap and decap cells are not reported as a separate class in `metrics.json`. Attempt 2 failed at 6.0 ns, so the design is only demonstrated at 6.5 ns even though the formula suggests a shorter period.

Q5. DRC and LVS. A DRC check verifies that the drawn geometry obeys the foundry's layout rules (minimum width, spacing, enclosure, density), so the chip can be manufactured reliably. An LVS check extracts a netlist from the layout and compares it with the synthesized netlist, to verify that the devices and connections drawn are the circuit that was designed. Both must pass: a layout can be DRC-clean but contain a short or an open that makes it the wrong circuit, and a layout that matches the netlist can still violate rules and fail in manufacturing. Our final run has 0 DRC errors (route, Magic and KLayout) and 0 LVS errors.

Q6. 8-bit multiplier. From the formulas of Section 2 (Table 8): full adders $N(N-1) = 8 \times 7 = 56$ instead of 12 (4.7 times more), AND gates $N^2 = 64$ instead of 16, and flip-flops $4N = 32$ instead of 16. The longest path is about $2N = 16$ full-adder delays instead of 8, so the delay doubles and the area grows roughly with N^2. Using the measured 2.063 ns of logic for 8 full-adder delays (0.258 ns each): $16 \times 0.258 = 4.13$ ns, plus 0.388 ns clock-to-Q and 0.134 ns setup gives $T_{min} \approx 4.65$ ns and $F_{max} \approx 215$ MHz (typical corner, ideal clock).

Table 8: Scaling from 4 bits to 8 bits (estimate for 8 bits).

Quantity	$N = 4$	$N = 8$
AND gates (N^2)	16	64
Full adders ($N(N-1)$)	12	56
Flip-flops ($4N$)	16	32
Critical path ($\approx 2N$ full-adder delays)	8	16
T_{min}	2.59 ns (measured)	about 4.65 ns
F_{max}	386 MHz	about 215 MHz

7 Conclusion

The 4-bit array multiplier passes all 256 test vectors at RTL and gate level, and the testbench was shown to detect an injected error. Synthesis produced 65 cells and 746.966 µm^2, with flip-flops taking 53.60 % of the area. Static timing analysis in the typical corner gave $T_{min} = 2.59$ ns ($F_{max} = 386$ MHz) with a critical path of 8 logic cells. Place and route needed three attempts: the slow corner forced a clock period of 6.5 ns, at which the layout is DRC-clean and LVS-clean with a setup slack of +1.519 ns, giving a post-route F_{max} of 200.8 MHz. The difference between Part C and Part D comes from the slow corner, wiring, and the clock tree and buffers added by the flow.